

e ID or Path

3. Enter Zuora Subscription ID or Name
4. Enter the Zendesk ticket link
5. Click 'Submit'

3.5 Reset Max Seats

Purpose: Reset the maximum seat count for a namespace

Steps:

1. Select Reset max seats tab
2. Enter Namespace ID or Path
3. Enter new maximum seat count
4. Enter the Zendesk ticket link
5. Click 'Submit'

3.6 Extend Subscription

Purpose: Temporarily extend soon-to-expire subscriptions

Current Limitation: If the namespace has been associated with a trial, you'll be redirected to extend the trial to provision a temporary extension

Future Development: Support for extending subscriptions for namespaces without previous trials is being implemented in Issue 12874

Future improvements / Bug fixes

For future improvements or bug fixes, please create issues in CustomersDot project, and tag the appropriate team:

1. Trial, license & namespace tooling > Provision
2. Consumables & max seat tooling > Utilization
3. Billing account/contact & QSR tooling > Subscription management

Action plan for Fulfillment-impacting bugs

If a bug is discovered that impacts Fulfillment, including provisioning, purchasing, billing, subscription data, etc., please do the following:

Reporting the issue

1. Open a new Fulfillment Meta bug intake issue (select the bugintake template) outlining what is known about the bug.

1. Tag Fulfillment Product Management by mentioning the @fulfillment-group/leadership/fulfillment-pm group.

1. Assign the issue to ofernandez2 for review and action.

1. Post the link to the issue on Slack in sfulfillment and in business-fulfillment-sync for broad awareness and review.

Notifying appropriate DRIs

The following individuals should be looped into the issue, depending on the impact of the bug:

- 1. For billing and revenue-impacting issues: Sarah McCauley - Director, Billing & AR
- 1. To determine impacted subscriptions and/or automation of Zuora solutions: Jessica Salcido - Finance Systems Administrator
- 1. For bookings impact, and/or to assess and coordinate Salesforce solution needs: Jesse Rabbits - Sr. Manager, Deal Desk
- 1. If customer outreach is needed, PM should work with: Lyle Kozloff - Director of Support, Global Readiness
- 1. For product monetization decisions needed: Justin Farris - Sr. Dir, Product Monetization

Fulfillment debugging and FAQ

License Activation issues

License won't activate due to a true-up or seat overage mismatch

- 1. For customers on 14.3+ we allow activating a license even if there's a 10% seat overage issue 333851. You can learn more about this [here](#) and see the associated MR 67507 and MR 67508
- 1. With Cloud Licensing, we won't block activation due to mismatch seats. If a customer applies a license with less seats than their currently active users, we will reconcile it in their following QSR or true-up event.

What type of connection does the GitLab instance require to activate Cloud Licensing?

The instance would need to have a 443 port connection to customers.gitlab.com in order to activate. This is also used for license synchronization as outlined in our documentation [here](#).

Data collection

Can customers opt out of telemetry or sharing license sync data?

The data transmitted with Cloud License is covered in this documentation. In short, it's aggregate user counts and some license metadata. This data is required for Cloud Licensing. It's intended to only include necessary data to support our needs for administering a license, supporting future renewals, supporting add-ons, and any seat reconciliations.

You can look at sample code that generates the counts by searching for subscription events in metrics.gitlab.com.

What is Operational vs Optional Data?

Our service usage data primarily aggregate counts from your instance (e.g. counts of issues or MRs) and is sent to GitLab on a weekly (or slower) cadence.

- 1. Operational Data: This is the data that is tied to your subscription account/license. Our goal with this data is to capture aggregate counts to help the GitLab Customer Success and Sales teams help customers better adopt GitLab and get value out of their investment. Those

counters can be viewed on metrics.gitlab.com by searching for "operational" and looking for the operational category. These operational events are agnostic of how you've activated your instance.

2. Optional Data: Optional Data is a similar type of metric (aggregate counts) but a much larger set than the Operational metrics. This is configured/opted-out by you, in your instance, and has nothing to do with your subscription. If you choose to share that data our Product teams would appreciate the insights to understand where to invest R&D resources to continue to mature our platform. Otherwise, you can ignore for your renewal and follow the opt-out steps documented [here](#).

Can a customer send subscription data ad-hoc, while keeping their GitLab instance airgapped/not connected to the internet?

Please see [Offline Cloud Licensing](#) for more information.

Fulfillment Roadmap Prioritization

Principles

Across our stable counterparts, we follow four key principles to keep us focused on delivering the right results. These principles are not absolute, the intent is for them to guide our decision-making.

Make conducting business with GitLab seamless

When customers choose to purchase GitLab they've already decided to unlock additional value by accessing the features or services enabled by a transaction. We strive to make the transaction experiences fade into the background, helping customers unlock this additional value as easily as possible. This creates a better customer experience and results in accelerated growth for GitLab.

This means that in every initiative we question the need for complexity. We strive to build functionality that is easy to understand and use, and make sure it works flawlessly for customers of all types. As much as we can, we won't require a customer to speak to a sales representative and will allow them to choose whether to transact via online self-service tools.

Build a strong foundation so GitLab can scale

Fulfillment systems are the foundational layer for many commerce activities within GitLab. Our systems provision licenses for customers, are the source of data for multiple KPIs and data models, and interact directly with Zuora and Salesforce. These systems need to be reliable, scale with demand, and allow other teams to collaborate.

We regularly invest in our foundations and will continue to pause new feature development in favor of foundations whenever we feel that our foundational systems aren't robust enough. We established a Fulfillment Platform group in FY23 for focused efforts in this area.

Use data to make decisions and measure impact

We have many sensing mechanisms at our disposal: feedback routed via our GTM teams, meetings

with business counterparts, customer feedback from user research, and improvement suggestions raised by GitLab team members and members of the wider community in our issue tracker.

We're also improving how we use data as a sensing mechanism to set direction and prioritization. Understanding our funnel is paramount in building a seamless commerce experience for our customers. Fulfillment teams in collaboration with Growth are instrumenting each point in our transaction funnels so we can use data to inform our strategy and direction.

Iterate, especially when the impact of a change is sizeable

Iteration is one of the most challenging values to follow, especially within Fulfillment. Oftentimes our work needs to be bundled and aligned closely with external announcements or communications. Even so, we strive to break work down as much as possible and decouple functionality releases from broader announcements. Doing this expedites delivering value to our customers and the business.

Minimize and remove business logic from the GitLab application code

In the past, we have embedded significant business logic into the GitLab instance code directly. For example, we have logic in our licensing system that checks at the instance level whether the customer license should be activated based on licenses paid for, etc. This causes significant issues as we evolve our business policies, which we can't then reflect in past GitLab versions that we support.

We will minimize such logic and remove it from the application code whenever possible, seeking alternative solutions.

Prioritization Process

Our roadmap is prioritized and scheduled following our Project management process. We aim to update this roadmap every month as a part of our milestone planning process.

To request work to be added to the Fulfillment roadmap, please follow our intake request process. Changes in priorities of this roadmap follow our prioritization process.

The source of truth for our priorities for a given quarter are our Fulfillment OKRs.

By nature of our direction, Fulfillment works mostly on highly cross-functional projects where either or both of the following are true:

1. Many cross-team dependencies: project execution relies on collaboration with other teams, with significant coordination of time and resources required.
1. Downstream impacts: projects may change how other teams operate (e.g., Field Operations, Enterprise Apps, Billing) and may also impact the success of their efforts (e.g., achieving sales efficiency targets, accomplishing e-commerce conversion goals)

To focus on the most impactful work, Fulfillment's prioritization process seeks to:

1. Prioritize the highest ROI initiatives as measured by long-term impact on GitLab, Inc's value. (Note: "cost to build" is a key consideration in the I of the ROI calculation)

1. Provide future visibility into priorities to adequately plan cross-team and cross-functional resource needs.

1. Minimize waste and churn due to re-prioritization mid-execution.

Prioritization Criteria

A project will be prioritized in the Fulfillment roadmap based on the considerations below.

1. Revenue impact potential

1. Value of unlocking new sales channels

1. Sales efficiency improvements and reduction in time spent by a field team member to close a deal, this includes eliminating sales team involvement on most transactions

1. Conversion improvements

1. Risks and compliance issues (negative revenue potential)

1. Operational cost reduction

1. Reduction in support costs

1. Seamless transactions with GitLab, efficiency gains, and improved customer satisfaction.

1. Foundations to unlock opportunities

1. Value of new or improved data to inform future opportunities

1. System robustness to support 10x customers

1. Value of the foundational work to unlock other opportunities

1. Number of GitLab team members able to contribute towards e-commerce improvements

1. Work that will help us scale (support more customers, improve operations, simplify business processes, etc.)

1. Confidence level around the impact and solution

1. Low for initiatives that haven't been properly scoped or researched. PM/UX/Eng and cross-functional partners will increase the confidence by scoping the initiative.

1. Ease of implementation

1. Consider the time and resources required to complete the initiative.

1. Consider a solution that is long-term sustainable, and corresponds to the revenue/cost impact estimated.

1. For efforts that are not well understood, we will start by assuming a larger effort to account for unknowns. As we do some scoping, we can refine the cost/complexity.

All initiatives, regardless of who requests them, will be evaluated based on this same criteria.

Some initiatives will have a direct impact on these criteria, but others will have an indirect impact. We will consider indirect impact as part of the prioritization.

When scoping new solutions we will prefer those that best allow GitLab to scale and accelerate future work. These solutions often require more upfront foundational work, which we will include in the initial scope. In cases when we decide to accelerate a solution by skipping on some foundational work, we will add this foundational work as a separate line item to the roadmap.

A note on Customer Satisfaction: to understand the impact of efforts aimed at improving customer satisfaction, we should estimate the indirect impact of improving CSAT on revenue and cost. For example, by reducing the number of steps or improving the steps required to purchase we will see an increase in conversion rate and thus revenue.

Scheduling new work

Prioritization based on the established criteria will drive the order in which work is scheduled to be completed. The product team will review overall prioritization regularly. Before changing priorities, will consider:

1. Efficiency and morale impact of disrupting ongoing efforts
1. Impact of changes to existing customer and partner commitments
1. Feedback from cross-functional partners

To minimize impact and give more predictability to partner teams, we will minimize changes to initiatives that we've already agreed with cross-functional partners to do within the ongoing quarter.

Anyone can request new items to be added to the roadmap via an intake request.

Quarterly cross-functional review

One of our prioritization goals is to maximize overall team output across Fulfillment and cross-functional partners. We want to give transparency to all GitLab team members about the work that Fulfillment and its partner teams plan to deliver.

To enable this, we will do a roadmap review with our stable counterparts before the beginning of a new fiscal quarter. As part of this review, we gather feedback on roadmap priorities, update the roadmap based on the feedback, and agree with partners on the scope and delivery milestones for the upcoming 3-6 months.

During these quarterly reviews we will aim to commit up to 70% of Fulfillment's engineering capacity for the upcoming quarter, and no more than 30% of capacity for the quarter after. This is meant to provide enough visibility into upcoming activities for cross-functional partners to plan for them while leaving room for reprioritization and changes as needed.

Communicating roadmap changes

Any proposed changes to the roadmap will be first communicated to cross-functional partners async in a relevant Slack channel with the relevant context and rationale, and ask for feedback. As needed, a synchronous meeting will be scheduled to discuss. All feedback will be considered by the product team and a final decision will be made and communicated once made.

Fulfillment Prioritization Escalations

If you have an urgent Fulfillment need that is not being prioritized by Fulfillment's regular prioritization process due to competing priorities, and it is both important and urgent, please work with your division's leadership for escalation. The first step can be an async discussion involving Fulfillment leadership (as of now include ofernandez2). Situations in which competing company-priority projects would need to be reprioritized to accommodate your request may require escalation to GitLab's e-group.

Team members in our Sales and Customer Success groups should escalate first via the Top ARR Drivers meeting for cross-team leadership visibility and prioritization. Simply add your item

to the list of asks (link in the meeting agenda) for discussion.

Fulfillment OKR processes

Fulfillment resourcing is allocated to work on top priority work, which is generally managed by:

1. First ensure that forced prioritization items (such as meeting SLOs) are completed timely.
2. Deliver on quarterly objective and key results (OKRs).

Fulfillment FY25 Q2 OKRs are listed in <https://gitlab.com/gitlab-com/gitlab-OKRs/-/workitems/6895> (team member access only). We have a similar top-level objective for the Fulfillment Stage each quarter.

Setting OKRs:

1. Each quarter the Fulfillment Stage product leader kicks off OKR planning by starting a top-level objective in GitLab OKRs with a draft list of top priorities.
1. Quads for each Fulfillment group work together to plan their quarterly work and add sub-objectives/KRs to map to those top priorities.
1. Quads suggest changes to OKRs based on what they are seeing during planning. This includes prioritizing tech debt, architecture blueprints, maintenance KRs, or any other work that may be missing from OKRs.
1. We label all sub-objectives or KRs using priority labels. We strive for 90%+ accomplishment on Priority 1, 80%+ on Priority 2 items, and 70%+ on Priority 3 items.

Reviewing OKRs:

1. Monthly updates to OKRs as part of what used to be the PI review meeting (example issue <https://gitlab.com/gitlab-com/Product/-/issues/13333>)
2. We make sure that KRs are up to date, including closing out and replacing OKRs when appropriate (e.g., we had a placeholder OKR that we decided not to pursue, or we decided to do a mid-quarter change to prioritize something else).
3. If everyone gets their updates in 2 days ahead of the PI review meeting, and there are no open discussion points after async review, we cancel the PI sync meeting.
4. Our agenda doc is updated each month, though, as we review async to ensure that all things that need conversation are discussed async. (agenda doc)

Finalizing and closing OKRs:

At the end of the quarter, stage leaders score the OKRs and leave a comment in the top level objective summarizing it. (FY25 Q1 example). This final step is critical since it helps us reflect and hold ourselves accountable, as well as celebrate what we accomplished which otherwise often goes unnoticed. We cross-post this last update to Slack for more visibility in our sfulfillmentfyi channel.

SECTION: product/groups/product-technical-program-management.md

Mission

Our mission as Technical Program Managers is to own the execution of complex, cross-functional initiatives that better enable GitLab to meet company objectives. We aim to make complex programs simpler with proactive, transparent, and efficient practices, while adapting our methodologies to the scenario at hand. We aspire to be trusted partners to stakeholders across the company, in order to accelerate and improve upon program outcomes over time.

Long-Term Vision

Define and consistently deliver against a standard of excellence for Program Management at GitLab.

Have a team reputation as a fundamental and necessary resource when R&D needs to deliver against complex business objectives and ultimately results for customers.

TPMs are seen across our company-wide stakeholders as a key element to accelerate GitLab's strategic business objectives.

Near-Term Goals for FY26

Excel in strategic program delivery; focus on a core group of highly visible and impactful programs, setting the tone for how our expanded team delivers value.

Create scalable workflows and processes that have a positive impact on GitLab's overall efficiency and solicits positive sentiment from stakeholders.

Continue to build our TPM team identity as the team expands; develop a strong team foundation and niche within the business. In doing so, evangelize TPM best practices.

Core Responsibilities

More information can be found in the Product TPM Job Family and the Product TPM Leadership Job Family

How We Work

Communication

Slack is the best way to reach out to our team; our goal is to respond to all requests within 24 hours. The best way is to send us a message in the technical-program-management slack channel.

Alternately, if an issue exists for a scope of work, you can tag us directly there.

Working hours

Our team is (as of this writing) distributed across the United States and can accommodate requests during US Business Hours - both east and west coast. We are looking to expand our team's time zone coverage to EMEA.

Many of our TPM's schedules are more flexible and accommodate hours outside of the hours noted above.

Async vs. Sync

Generally, our team will default to async communication in alignment with GitLab values. When it is needed, we will pull team members into a call or meeting to ensure we are working effectively and efficiently if needed.

Cross-functional initiatives typically need more sync time. If you are part of a core project team, we will likely find recurring meeting time to drive execution for the length of the project.

How You Can Leverage The Product TPM Team

If you have an initiative that you think could benefit from program management support, the best first step is to message technical-program-management following this template to intake a request.

Work that can benefit from program management support is typically complex, cross-functional, and critical to the business. We determine if a project is a good match for TPM support with the TPM Coverage Calculator ([internal only link](#))

What about Program Managers outside of the Product Org?

As Technical Program Management is a relatively new function at GitLab, we're continually iterating on how the various TPM and PgM teams collaborate.

|Product|Infrastructure|Security|

|:-----|:-----|:-----|

|The R&D Technical Program Manager role supports cross-division initiatives across PM, UX, Engineering and other functions within R&D along with functions across GTM| Infrastructure Technical Program Management Team drives the planning, execution, and delivery of complex infrastructure projects across Engineering and Product.|The Security Program Manager will be responsible for complete overview and driving security initiatives across product, engineering and business enablement.|

|Manage and collaborate on key initiatives with stakeholders and leadership across R&D. Drive accountability ensuring all program objectives are met.|Establish a reliable and visible cadence for initiative reviews, decision making, prioritization, and staffing management.|Communicate successfully with partners and drive accountability across teams to ensure program objectives are met |

|Refine and optimize division processes improving overall efficiency and execution of the division.|Lead and drive Infrastructure projects such as: Availability, Disaster Recovery, Releases, CI/CD Pipeline Efficiency, and Engineering Productivity|Gather and examine customer and internal security team needs and opportunities for new information security programs, products and projects |

|Gather inputs and feedback and suggest improvements to our Product Process, and operational model within the Product division.|Provide technical project management, coordination, and inter/intra team communications to deliver outstanding program outcomes.|Drive organizational needs cross-functionally with product, engineering and finance including tracking and reporting of progress and metrics |

| Product Technical Program Management | Infrastructure Program Management | Security Program Management |

Contacting The Team

The best way to reach out to our team is through technical-program-management slack channel.

Why use the channel and not direct message a PM I already know?

Even though this can seem like a good idea, it limits visibility to the larger TPM team and doesn't allow for proper resource allocation. Sending your requests in this channel will allow for full visibility and ensure our team is connecting you with the program manager who is a good fit for your project and has bandwidth to take on something new.

SECTION: product/groups/project-management.md

Plan: Project Management {welcome}

View all team members and stable counterparts

The responsibilities of this collective team are described by the Project Management Group. Among other things, this means working on GitLab's functionality around work items, boards, milestones, iterations, to-do list, time tracking, planning analytics, and notifications.

- I have a question. Who do I ask?

In GitLab issues, questions should start by mentioning the Product Manager (@gweaver). For UX questions, mention the Product Designer (@nickleonard). GitLab team-members can also use splan.

Direction

GitLab > Dev Section > Plan Stage > Project Management Group

Performance Indicators

Customer Value

- Paid Monthly Active Users (Paid GMAU)
- Monthly Active Users
- Team Planning and Planning Analytics Category Maturity
- System Usability Score (SuS) - Decrease the count of detractors attributable to the Project Management product surface area on a rolling quarterly basis

Product Quality

- Target Error Budget of > 99.95%
- Escaped defects - The count of bug or security issues filed for defects or vulnerabilities found on canary or production on a rolling monthly basis

Process

- Open MR Age (OMA)
- Open MR Review Time (OMRT)

- Merge Request Rate - Average MRs per engineer on a rolling monthly basis
- Lead Time - The median number of days it takes for an issue to flow through workflow::validation backlog to closed.
- Validation Track Cycle Time - The median number of days it takes for an issues to flow through workflow::validation backlog to workflow::planning breakdown.
- Build Track Phase 1 Cycle Time - The median number of days it takes for an issues to flow through workflow::planning breakdown to workflow::ready for development.
- Build Track Phase 2 Cycle Time - The median number of days it takes for an issues to flow through workflow::ready for development to closed.
- Adoption of Product Development Flow workflow labels

History of Process Improvement Efforts

Goal	Status	Issue
> 90% of issues correctly reflect their current Product Development Workflow stage	In Progress	<https://gitlab.com/gitlab-org/plan/-/issues/442>
Engineering committed issues in the current release have the Deliverable label applied by the 17th each month	In Progress	<https://gitlab.com/gitlab-org/plan/-/issues/442>

How we work

- In accordance with our GitLab values.
- Transparently: nearly everything is public, we record/livestream meetings whenever possible.
- We get a chance to work on the things we want to work on.
- Everyone can contribute; no silos.
- We do an optional, asynchronous daily stand-up in splanstandup.

Capacity Planning

When we're planning capacity for a future release, we consider the following:

1. Availability of the teams during the next release. (Whether people are out of the office, or have other demands on their time coming up.)
1. Work that is currently in development but not finished.
1. Historical delivery (by weight) per group.

The first item gives us a comparison to our maximum capacity. For instance, if the team has four people, and one of them is taking half the month off, then we can say the team has 87.5% (7/8) of its maximum capacity.

The second item is challenging and it's easy to underestimate how much work is left on a given issue once it's been started, particularly if that issue is blocking other issues. We don't currently re-weight issues that carry over (to preserve the original weight), so this is fairly vague at present.

The third item tells us how we've been doing previously. If the trend is downwards, we can look to discuss this in our retrospectives.

Subtracting the carry over weight (item 2) from our expected capacity (the product of items 1

and 3) should tell us our capacity for the next release.

Workflow

Issues and epics generally follow our Product Development Flow.

Starting in January 2022, we will be running a 3-6 month experiment to shift the planning cadence from milestones to iterations with the primary goal of planning in smaller batches to enable more timely, better decision making. Iteration planning will take place via our 30 minute weekly Engineering/Product/UX sync. Only issues that have been weighted and marked as ~workflow::ready for development will be scheduled into upcoming iterations. While we will be leveraging iterations, we will still follow our documented product development timeline

Project Management Boards:

- Validation Track
- Build Track
- Iteration Planning
- Milestone Planning
- Team

Themes

A small number of high priority features will be chosen as 'themes' for a period of time. Themes provide an opportunity for the whole team to rally around a deliverable, even if they don't contribute directly to it. These items are given especially close attention by all those involved with a view to delivering small iterations and keeping work unblocked. There should never be more than two themes in progress at a time per team.

- A Slack channel is created with the convention f[feature name].
- An epic hierarchy is created with sub-epics mapping to iterations, each achievable within a milestone.
- Iterations are broken into multiple issues that can be accomplished independently, and PMs schedule those as normal.
- Other actions may be established, such as regular 'office hours' calls.

Team-members work together to continuously refine the iterations as complexity is revealed.

Examples of successful themes:

1. Requirements Management (frequirements-management, Epic)
1. Jira Importer (fjira-importer, Epic)

Talking With Customers

In a perfect world, we would have cross-functional representation in every conversation we have with customers. To help work towards realizing this, anyone who is scheduling a call with a customer via sales, conducting usability research, or generally setting up a time to speak with customers or prospects is encouraged to add the Plan Customer Interviews calendar as an invitee to the event. This will automatically populate the shared calendar with upcoming customer and

user interactions. All team members are welcome and encouraged to join -- even if it's just to listen in and get context.

You can subscribe to the calendar and invite it as a participant in a customer meeting that you are scheduling using the URL [gitlab.com5icpbg534ot25ujlo58hr05jd0@group.calendar.google.com](https://calendar.google.com/calendar/invite/gitlab.com5icpbg534ot25ujlo58hr05jd0@group.calendar.google.com).

Retrospectives

The Plan stage conducts monthly retrospectives in GitLab issues. These are confidential during the initial discussion, then made public in time for each month's [GitLab retrospective]. For more information, see [team retrospectives].

The retrospective issue is created by a scheduled pipeline in the [async-retrospectives] project. For more information on how it works, see that project's README.

- GitLab retrospective
- team retrospectives
- async-retrospectives
- retros

Product Shadowing

Engineering team-members can shadow a product stable-counterpart. Shadowing sessions last two working days, or the equivalent split over multiple days to maximize experience with different functions of the role. To shadow a counterpart on the team:

1. Create an issue in the plan project tracker using the Product-Shadowing template;
1. Create a WIP MR to this page to update the table below, adding your name and issue link, and
1. When your counterpart is assigned to the issue, add their name, remove WIP status and assign to your manager for review.

Month	Engineering counterpart	Product counterpart	Issue link
----	-----	-----	-----
2020-07	Charlie Ablett (@cablett)	Keanon O'Keefe (@kokeefe)	gitlab-org/plan118
2020-10	Jan Provaznik (@jprovaznik)	Gabe Weaver (@gweaver)	gitlab-org/plan185

Speed Runs

- Labels
 - Scoped Labels
- Issues
 - Description Change History

SECTION: product/groups/verify.md

Verify

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/9iF9zWAxdH0" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

Intent of this page

The Verify Product Group wants to increase sharing, findability, and encourage bias for async communication. We will use this handbook page as forum for showcasing our team processes while also providing an opportunity to encourage a open discussion between team members on our processes for retrospective.

Where can I reach you?

Slack: sverify

GitLab Project: Verify

Verify teams:

- Verify:Pipeline Execution
- Verify:Pipeline Authoring
- Verify:Runner
- Verify:Pipeline Security

What do y'all do?

The purpose of this product group are described by the Verify stage direction. This is inclusive, though not limited to, the pipeline experience, creating or authoring .gitlab-ci.yml files, executing jobs in a pipeline, and various testing capabilities in CI/CD.

Continuous Integration and Runner Boundaries (Ownership/DRI's)

While generally the functions of Continuous Integration and Runners are interdependent (runners are the build agents that execute CI jobs), as Verify categories they represent distinct feature areas. However, it is sometimes difficult to discern which team or product manager is the DRI for a feature or capability that doesn't seem to fit neatly within the high-level categories and this section aims to clarify those boundaries.

At GitLab, using RADICIE to assign responsibilities means everyone is Consulted and Informed, but to highlight critical team members who must be consulted the table below specifies who explicitly.

The noted DRI still owns decision-making and is still responsible for notifying/consulting critical team members, however the 'Must be Consulted' designee(s) takes the guesswork out of who to involve when these team members may not be part of the core engineering team aligned to the DRI product manager.

Category	Code Base	Description	PM DRI	EM DRI	Must be consulted
Runner Core	runner	Development of features for the core runner code base for use by self-managed customers on various compute platforms and architectures (runner binary, docker images)	Runner PM - Darren Eastman	Runner EM - Nicole Williams	N/A
Runners Fleet	RAILS	Development of features for the configuration, use and administration of runners in the GitLab UI.	Runner PM - Darren Eastman	Runner EM - Nicole Williams	tbd
Runner SaaS	autoscaler	SaaS Runners (Linux, Windows, macOS) and internal GitLab Runner fleet	Runner PM - Gabriel Engel	Runner EM - Nicole Williams	Infrastructure Mgr. - David Smith
Compute Minutes Management for SaaS runners	RAILS	Changes to compute minutes configuration for customers on GitLab SaaS in the scope of SaaS runners.	SaaS runners PM Gabriel Engel	Runner EM - Nicole Williams	N/A
CI Pipeline Execution and Performance	RAILS	Features and capabilities for running or triggering a pipeline in the GitLab UI.	CI PM	CI EM - Cheryl Li	N/A
CI/CD Queueing GitLab SaaS and Self-Managed	RAILS	CI jobs queueing architecture.	Pipeline Execution PM	Lead engineers: Kamil Trzeci-ski , Grzegorz Bizon, Stan Hu	
CI/CD Fair Scheduling GitLab SaaS	RAILS	CI jobs fair scheduling architecture and configuration.	Pipeline Execution PM	Lead engineers: Kamil Trzeci-ski , Grzegorz Bizon, Stan Hu	

Workflow

We work in a Kanban-style aligning with Milestones and GitLab's Product Development Flow.

Cross-Functional Partnership

We work across other teams often and are striving to get better at engaging with others. If you are interested in being a part of this dialogue - drop a line our sverify slack group!

Product Marketing Enagement

We partner really closely with Product Marketing Management. In the Verify Stage, we have a stable counterpart assigned, as defined in the PMM Team Structure, which aligns to the CI Use Case.

We have four main processes:

1. Semi-annual Stage-wide GTM & Strategy Positioning
1. Quarterly Sales Enablement
1. Quarterly Roadmap Alignment for Product Groups
1. Monthly Release Post and Market Positioning for Product Groups

Stage-wide GTM & Strategy Positioning

- Frequency: Twice a year
- Audience: Internal
- Assets Produced: Deck, Recording, and Issue Boards
- Goal: Deliver a 30-minute recording that aligns the half-year product strategy of the stage with the GTM sales motion and competitive landscape. Include any assets that will be helpful to sales teams for pitches and product managers for early selling in prospect calls.

We recently conducted our first joint stage-wide PMM and Product Positioning for FY22 1H, alongside a deck.

Sales Enablement

- Frequency: Quarterly
- Audience: Internal
- Assets Produced: Deck and Recording
- Goal: Deliver 10-30 minute recordings on various product topics for use in field enablement.

We recently participated in a discussion of building the bench for technical topics via cs-skill116. These types of sessions would then be used in partnership with PMM to help build compelling stories for the Field.

Roadmap Alignment for Product Group

- Frequency: Quarterly
- Audience: External
- Assets Produced: Recording
- Goal: Deliver 10-30 minute recordings on various product topics for use in webinars, partner calls, or whitepapers.

These are intended to help guide themes and present opportunities to build stories in the market and not commitments, otherwise, these would be scheduled for a milestone.

Release Post & Market Positioning for Features

- Frequency: Monthly
- Audience: External
- Assets Produced: Copy/text
- Goal: Deliver written communication in Merge Requests with Product Manager in the Release Post Items or blog posts.

We sometimes ship feature specific blog posts such as Moving CI to Lovable...again or this expose on the Pipeline Editor. These are great opportunities to include PMM alongside the Release Post review and positioning.

Customer Success & Renewal Engagement

We have a CS <> Product feedback loop for Customer Success (SA/CSM) and Product Managers to sync on product feedback from prospects and customers. On a monthly cadence, we meet to discuss problems to solve as captured in issues created in the cs-product-feedback project and consolidated on this issue board.

Alliances Engagement

More details to come

Community Contributions

We want to encourage and support our open source community as much as possible. We have two measures of success:

1. Merged MRs from the community in the Verify Stage
1. MRARR

Our process for enabling merge requests from the community can be found on the Verify Team Page.

SECTION: product/groups/growth/_index.md

Overview

The Growth stage is responsible for scaling GitLab's business value. To accomplish this, Growth analyzes the entire customer journey from acquisition of a customer to the flow across multiple GitLab features - and even reactivation of lost users. Several groups help with this mission:

- Activation, Conversion, Expansion, and Adoption connect users to the existing value that GitLab already delivers by rapid experimentation.
- Analytics Instrumentation builds the backbone of data that other groups need to be successful, enabling a data-informed product culture at GitLab.

Growth's ultimate goal is to connect GitLab's value as a Single DevOps Platform with our customers. In order to do that, we take a zoom in and zoom out approach. We break down the entire GitLab growth model, and identify the highest ROI lever at this moment to focus on. In the Growth direction page we outline the Growth section's long term direction and near term focus areas.

How the Growth section works

All the Growth team members are listed here
We follow the Product Development Flow.

Growth ideation and prioritization

Anyone is welcome to submit ideas to the experiment idea backlog using the experiment idea template.

To prioritize, the growth groups will use the ICE framework, which consists of the following elements, scored on a scale of 1-10:

- Impact - high score for high impact
- Confidence - high score for high confidence
- Ease - high score for ease

Weekly Growth meeting

The Growth Product Management Director should run a weekly growth meeting to review, prioritize, and plan experiments. The meeting should take place on Tuesdays for 50 min and should cover the following agenda.

5 min: Social questions

5 min: Announcements

30 min: Group updates

- KPI & OKR (Live)
- Highlights & Learnings (Live)
- Activities (Read only)
- Next UP (Read only)

10 min: Discussions

To prepare for the meeting, the Growth PM's and the Growth Director should check in on experiments to identify which can be concluded, and to collect data for review.

Other regular Growth meetings

In addition to the weekly Growth meeting, Growth team members participate in the following regular meetings:

1. Weekly Growth Engineering

- 1. Attendees: Growth team members
- 1. Goals: Discuss anything Growth engineering related.
- 1. Agenda: [link](#)

1. Weekly Analytics Instrumentation and Data Weekly Sync

- 1. Attendees: Analytics Instrumentation and Data
- 1. Goals: Stay in-sync for work across Analytics Instrumentation, Data Engineering, and Data Analysis.
- 1. Agenda: [link](#)

1. Weekly Growth & Threat Management EM

- 1. Attendees: Growth & Threat Management EMs
- 1. Goals: Stay in-sync on growth engineering efforts.
- 1. Agenda: [link](#)

1. Weekly Growth PM sync

- 1. Attendees: Growth PMs
- 1. Goals: Stay in sync as a PM team, discuss PM specific issues/ideas
- 1. Agenda: [link](#)

1. Analytics Instrumentation, Fulfillment, Data, Customer Success, and Sales Weekly Sync

- 1. Analytics Instrumentation, Data, Customer Success, and Sales
- 1. Goals: Stay in-sync for work across Analytics Instrumentation, Data, Customer Success, and Sales
- 1. Agenda: [link](#)

1. Monthly sync meeting with the Data team

- 1. Attendees: Growth PMs and Data team
- 1. Goals: Keep the Data team aware of upcoming changes that may impact them, get proactive feedback on how to set up data and tracking needs
- 1. Agenda: link
- 1. Monthly sync with Growth Marketing
 - 1. Attendees: Growth PMs and Growth Marketers
 - 1. Goals: Stay in-sync and work as efficiently as possible together
 - 1. Agenda: link
- 1. Monthly sync with Product, Legal, Compliance, Data, and Privacy
 - 1. Attendees: Product, Legal, Compliance, Data, and Privacy
 - 1. Goals: Answer any legal, compliance, and privacy concerns
 - 1. Agenda: link
- 1. Monthly sync with Growth section and Application Security
 - 1. Attendees: Product, Engineering, Application security
 - 1. Goals: Growth / Application Security stable counterparts monthly discussion
 - 1. Agenda: link

How Growth launches experiments

Since the Growth section is among the first groups to launch product experiments and A/B testing, we summarized the current tooling and process in the list of guides and documentation below to help other teams interested in experimentation to get started faster.

- Experiment guide for Product managers
- Experiment ideation Process
- Engineering guide for how to run experiment
- Growth Engineering Handbook page on running experiments
- A way for GitLab team members to view currently active experiments on GitLab.com

How Growth collaborates with other groups

The hypotheses explored by Growth will span across groups and stages. How do other groups interact with Growth?

Growth does not own areas of the product, but finds ways to help users discover and unlock value throughout the GitLab application. They will do a combination of shipping low-hanging fruit themselves, but will partner with other stages on larger initiatives that need specialized knowledge or ongoing focus.

It is the responsibility of other stages to maintain and expand core value in their respective product areas; it's

Growth's mission to streamline delivery of that value to users. For example:

- Manage is responsible for user registration and management. This stage maintains a long-term vision on how users are created and maintained in GitLab, with a roadmap that includes other categories and priorities.
 - Growth may identify user registration as an opportunity to further the group's priorities.
- Growth's

engineering team may ship smaller improvements independently of Manage's direct involvement during implementation (such as soft email confirmation), but may need to partner with Manage on larger efforts like a completely new onboarding experience.

- The group ultimately owning the change - in this case, Manage - would review Growth's contributions into their area of the product
- and - like a contribution coming from the wider community - ultimately own the final result.

UX

How UX participates in milestone planning

When the planning issue is created the Product Designer should follow these steps:

- Review the issue boards and the prioritization dashboard (issues should be ordered by priority)
- Determine whether they agree with the prioritized issues or want to suggest a change. For example, are there any SUS impacting issues we've been neglecting?
- Comment in the planning issue to discuss proposed changes.
- Re-order the board once priority is decided.
- Ensure that issues have weights.

A note about capacity management

Every milestone will have more work than we can commit to. Product designers know the approximate number of issue weights they can take on in order to do the work to our high standards. Never take on more issue weights than you can, and don't compromise other important tasks such as professional development and UX department initiatives. This leads to burnout. Follow the escalation instructions to make your wider team aware of the issues you can't take on so that they can explore additional options or tradeoffs.

How UX Works

We follow the Product Designer workflows and UX Researcher workflows described in the Product Design section of the handbook. As Growth designers, we relentlessly measure the impact of our design changes following the experimentation workflow. In addition:

- we have issue boards so we can see what everyone is up to. Refer to issue boards in our planning issues. For example, this is the template for Acquisition Planning issues.
- we label our issues with UX, devops::growth and group::.
- we will label experiments with UX problem validation and UX solution validation according to the UX Research Workflow definitions to indicate the type of learning the experiment achieves. The purpose of these labels is to track this UX KPI related to research velocity.
- we use the workflow labels for regular issues and experiment workflow labels for experiment issues.
- we use milestones to aid in planning and prioritizing the four growth groups of Acquisition, Conversion, Expansion and Retention.
 - PMs provide an ICE score for experiments and by using priority labels for other issues.
 - The Product Designer applies the milestone in which they plan to deliver the work (1-2 milestones in advance, or backlog for items that are several months out. For example, if an issue is not doable for a designer in the current milestone, they can add the next milestone to

the issue, which will communicate to the PM when the work will be delivered.

- If the PM has any concern about the planned milestone, they will discuss trade-offs with the Product Designer and other Growth PMs.
- we use UX issue weights in order to better estimate capacity, realistically break down our work, and give PMs a little insight into how much work we can take on in a milestone.
 - Label the issue for UX work with UX and assess the issue weight. Issues larger than an 8 should be broken down further.
 - Use the scoped labels starting with design weight to add the UX weight to an issue.
 - When the UX work is ready to be transitioned into Engineering, apply the workflow label workflow::planning breakdown.
- We have a Figma template for designing experiments. You should title your Figma designs to be consistent with the experiment name, and link the experiment issue to the Figma file. When the variants are ready, add the control and variants to the "All the variants" page and provide context as needed. If you are conducting multiple experiments in the same area, consider using the same Figma file but include different pages per experiment design.

UX Definition of Done (DoD)

Together with the Product Manager, the Product Designer applies the DoD to epics in order to better break down design work and give counterparts better insight into which steps in the design workflow need to be completed before the MVC can move to the development phase.

In addition to the Validation Phase Outcomes listed in the Product Development Flow, we also make sure that:

- For an experiment, experiment issues have been created with a hypothesis and experiment plan, and experiment labels have been applied
- Cross-team dependencies have been identified and those teams notified and their feedback received
- Prototypes or mocks for the selected solution have been completed and it is clear through the documentation and content of issues that they are Ready for Development
- Previous versions and discussions of the solution(s) are labeled as 'Draft', and should not appear in the Epic description
- If changes involve copy, ~"Technical Writing" and ~"UI text" labels have been added
- Design spec has been added to the designs. Try to link to relevant components in gitlab-ui
- For design patterns that aren't in Pajamas (done sparingly), detailed implementation annotations are included engineering has reviewed
- Follow up issues have been created in the design system project for any new design patterns or documentation updates
- UX issues are closed and the SSOT (epic) is updated with a link to the completed design, along with design rationale for key decisions.
- The MVC engineering issue has been updated and labeled ~"workflow::planning breakdown"

Visual Reviews of MRs

The engineering team applies the UX label to any MR that introduces a visual, interaction or flow change. These MRs can be related to new issues, bugs, followups, or any type of MR. If the engineer isn't sure whether the MR needs UX, they should consult the designer who worked on the related issue, and/or the designer assigned to that stage group, or the Product Design Manager.

Visual reviews are required for any MR with the UX label. When the MR is in workflow::In review, the engineer assigns the MR to the designer for a visual review using the reviewer functionality in the sidebar. This can happen in parallel with the maintainer review, but designers should prioritize these reviews to complete them as quickly as possible.

There are times when it isn't possible or practical for a designer to complete their visual review via Review Apps or GDK. At these times the designer and engineer could coordinate a demo or the engineer could record a video of themselves going through the new functionality and add it to the MR. Creating a video is a way to speed up the review process, however it is optional and not always an appropriate stand in for a full review.

UX Scorecards

All of the planned, in progress and completed UX Scorecards for Growth can be found in this epic.

For more information, read about UX Scorecards.

UX Scorecards can be used to evaluate onboarding experiences and we also include onboarding criteria in our UX Heuristics.

UX Themes and Labels

We use labels to track UX improvements across a few themes. This allows us to track our work more holistically against big areas we've identified for UX improvement.

For now, these themes/labels are:

- Easy to Trial GitLab
- Feature Discoverability
- Onboarding New Users
- Easy Upgrade Flows

Collaboration process with other Product teams

We use this process to make sure growth and product teams are maximizing business impact while ensuring close collaboration.

Proposed Growth-Feature Product Owner collaboration process

1. Growth identifies a focus area that may potentially touch the areas owned by another product team. For example, growth will be focusing on new user adoption of Create & Verify features starting in FY21 Q4.

1. Growth reaches out to have a sync or async kickoff conversation with the product team (ideally including PM, EM, UX). The goal of the kickoff is to:

- Inform the Product feature owner that growth is planning to experiment in this area;
- Share growth's high level plans and ideas to avoid conflicting roadmap and duplicate efforts;
- Form a plan how the 2 teams can keep each other updated async, for example, ping the PMs

on the relevant issues async or sync meetings etc.

- Ideally, establish understanding of both teams' KPIs and objectives

1. Growth will post regular updates on what we are working on, what has been shipped, what's up & next, and the results & learnings from analyzed experiments to channels such as Key Reviews, GC, and potentially a monthly summary video.

1. Most growth experiments will be focused on UX/copy flow changes initially, and these will follow the typical GitLab code review process.

1. We don't expect to have many new "features" developed from growth experiments, but in the rare case that this happens or if we feel like the UX flow we developed needs to have a permanent owner, the Growth team will work to identify an appropriate product owner, and hand it off to the owner following the typical community contribution process.

1. For UX Collaboration, Product Designers will do the following

- The Growth Product Designer will mention the stage group Product Designer when starting work on a design issue for awareness and to gather background information such as existing constraints, usability problems and existing research. (Growth PMs will communicate in a similar way to Stage PMs)

- The Growth Product Designer will add the stage group Product Designer to design reviews to get feedback and ensure consistency. Design reviews can be sync or async, for example:

- Scheduling a recurring monthly sync call.

- Creating a document for async catching up (template) and a calendar event to remind everyone involved to share what they're working on.

- UX Researchers should share research results with stage groups whenever that research is relevant cross-stage.

Contributing to the Learn GitLab project

The Learn GitLab project is utilized to onboard new users to GitLab and contains self-paced issues the user can complete to help set them and their team up for success within the GitLab platform. There are currently two separate Learn GitLab projects, one for new free SaaS signups and one for new SaaS trial signups. The growth team is still actively experimenting with improvements to this experience. If you'd like to contribute an improvement to one of the Learn GitLab projects, you can do so by following the steps outlined in this issue.

Growth RADCIE and DRIs

In alignment with GitLab's RADCIE approach, the DRIs for the following tasks would be:

| Task | DRI |

| ---- | --- |

| Hypothesis Generation | Growth Product/Engineering |

| Experiment Design | Growth Product/Engineering |

| Experiment Implementation | Growth Product/Engineering |

| Contribution Review | Stage Product/Engineering |

| Experiment Ramp | Growth Product/Engineering |

| Post-Experiment Analysis | Data Team |

| Post-Experiment Decision | Growth Product/Stage Product/Engineering |

| Maintenance | Stage Product/Engineering |
| Alert creation | Growth Product/Engineering : How to create a Sisense SQL alert |

It is in the Growth teams purview to run any experiment in any area of the application. Growth's ability to experiment in this manner is designed to further learning potential for the larger GitLab team and support business priorities. When an experiment impacts the area of another product owner/team, Growth informs them following the collaboration model above. Product owners/teams are encouraged to raise concerns and provide further context. Ultimately, Growth determines whether the experiment is deemed a success using data and input from relevant teams. The product owner/team is made aware of the result and next steps.

SECTION: product/groups/product-analysis/_index.md

Product Data Insights Handbook

The Product Data Insights group consists of a team of product analysts. This group reports to the VP, Product Management and serves as a functional analytics team to support the GitLab R&D Organization (Product and Engineering divisions), in addition to product data-related analysis across GitLab.

In addition to supporting the Product and Engineering divisions, the Product Data Insights team is an active contributor to the GitLab Data Program. In addition, the Product Data Insights team is part of the Functional Analytics Center of Excellence (FACE), along with other functional analytics groups across the GitLab Data Program.

Read more about what we do at GitLab on our [Direction](#) page.

Team members

Product Data Insights is a small (but mighty) team. In order to support the Product and Engineering divisions, each analyst is assigned to one or more sections or teams to support.

Name	Title	Product Section or Team
Carolyn Braza	Senior Manager	Product Data Insights Analytics
Dani Deng	Senior Product Analyst	Growth, Engineering
Nicole Hervas	Senior Product Analyst	CI, CD
Lily Fisher	Senior Product Analyst	Infrastructure Platforms, Engineering
Emma Fergen	Staff Product Analyst	Data Science
Matthew Petersen	Senior Product Analyst	Dev
Dave Peterson	Staff Product Analyst	Sec
Neil Raisinghani	Senior Product Analyst	Fulfillment, Pricing

Handbook contents

- Dashboards, Analysis, & Insights
- Team Processes

- Product Data Insights Data Models Cheat Sheet
- Crash Course for Product Stage Resources (Including Section::Growth)
- Experimentation Design & Analysis
- dbt Cheat Sheet for Functional Analysts

Working with us

The Product Data Insights group works in two-week iterations, which dictate how and when we plan and prioritize work. Iterations start on Thursdays and end on Wednesdays.

You can see our current iteration [here](#).

Issue intake

For all Product Data Insights requests, please create an issue in the Product Data Insights project, apply the Team::PDI and product data insights labels, and follow the guidelines below.

All data issues with the Team::PDI label will appear on the Product Data Insights board.

Issue templates

Please select the appropriate template based on your type of request and answer as many of the questions as you can. The more information and context we have up front, the faster we are able to triage and begin work on the issue.

Request Type	Template
Ad hoc / Default request	Ad Hoc Request
Create new or update existing PI chart	PI Chart Help
Experimentation analysis	Experiment Analysis Request
Iteration planning	Iteration Planning

Submission due date

In order to be considered for the upcoming iteration, please open all issues by EOD Monday before the next iteration begins. We understand that urgent matters come up, but please try to adhere to the submission due date for any planned work.

Issue prioritization

The PDI team takes three different priorities into account when scheduling work, each captured by a scoped label:

1. section-priority::
1. pm-priority::
1. pdi-priority::

Section priority

Section priority is ultimately determined by section leaders and product leadership. It should align with overall product investments and business impact of the work. Analysts have default guidelines to assign section priority, which then becomes part of the prioritization discussion with section leadership.

Section priority labels will be added by product analysts or section leadership.

Label	Definition
---	---
section-priority::1	Work supporting high-impact, high-investment stages or groups; cross-stage analysis
section-priority::2	General work supporting the section; help with PI charts; etc
section-priority::3	Low-impact work; "nice-to-haves"

Most issues will fall under section-priority::1 and section-priority::2.

PM priority

PM (Product Manager) priority captures the relative priority of the issue compared to any other issues that the PM (or their group) have open in the backlog (if applicable). In general, the issues that are more immediately actionable and impactful to the company KPIs should be higher in priority.

We ask that PMs apply a pm-priority:: label to issues to indicate relative priority of the request.

Label	Priority
----	-----
pm-priority::1	High and/or urgent
pm-priority::2	Medium
pm-priority::3	Low, non-urgent

PDI priority

PDI priority is owned by the individual analyst. The Senior Manager, Product Data Insights will help refine priority based on importance and capacity. The team will work with the VP, Product Management and/or Product leadership on trade-offs (if needed).

Section priority and PM priority are both inputs in determining PDI priority. However, the scope of the team's work extends beyond section support (ex: cross-functional initiatives), and therefore there are other considerations, as well.

PDI priority labels will be added by the Product Data Insights team as a part of triage and planning.

Label	Priority
----	-----
pdi-priority::1	High / Urgent Priority Any analysis requests that are required to be completed within the current iteration. All requests that have Priority 1 should have a direct KPI and/or OKR that will be affected by the analysis.

| pdi-priority::2 | Medium Priority This is where most requests would fall into. This can be any net-new analysis, reporting (dashboard creation), or exploratory analysis that is needed for decision making. |

| pdi-priority::3 | Low Priority / Consultant: This is for any analysis that does not have an immediate direct action as a result of the analysis and/or other low-level, non-urgent requests that can be placed in an analyst's backlog. |

Most issues will fall under pdi-priority::2 and pdi-priority::3.

Iteration planning

Final commitment and prioritization will occur during the iteration planning meeting, which takes place the day before an iteration begins (every other Wednesday). The team will consider new and existing issues, along with issues in progress. When selecting issues for the next iteration, the team considers the following:

- Issue priority
- Issue weight
- Velocity
- Working days
- Target work breakdown

Issue priority

Analysts will use the different priority labels (as defined above) as inputs into planning.

Issue weight

Each issue is assigned a weight based on estimated time commitment.

If a single issue has a weight greater than the length of the iteration (2 weeks), it should be broken into smaller units of work. (This could also be an indicator that the issue should be converted to an epic).

| Weight Value | Estimated time to complete |

| --- | --- |

| 1 | < 1 hour |

| 2 | 2 hours |

| 3 | 4 hours |

| 5 | 1 day |

| 8 | 2-3 days |

| 13 | 1 week |

| 21 | 2+ weeks |

| 34 | 1+ month |

Velocity

Product Data Insights defines velocity as the amount of work (measured in issue weight) completed

by the team within a given iteration. While we recognize that this is an imperfect measurement

(partially-completed issues and undocumented work are not accounted for), it is a rough gauge of team output.

We aim to only commit to work we believe can be completed within the 2-week iteration. As such, we will commit to less than our recent velocity and leave a buffer to account for urgent issues and interruptions. To start, each analyst will leave a buffer of ~2 days worth of work (an estimate based on the recent volume of unplanned work). High-priority issues exceeding the allotted buffer will have a material impact on our ability to complete planned work, so please plan ahead if you know that you will need assistance from the Product Data Insights team.

Working days

As GitLab team members, we are encouraged to take PTO and observe public holidays in order to maintain a healthy work-life balance. Analyst capacity should be adjusted based on the number of days they are working in the iteration.

Target work breakdown

Product Data Insights groups work into three different buckets. During planning, analysts will aim to maintain the following breakdown:

- Section-level priorities: 50%
 - Backlog DRI: Section leader and/or product leadership
 - This work can be identified with the section-priority::1 label
- Other section work: 25%
 - Backlog DRI: Product analyst
 - This work can be identified with the section-priority::2 or section-priority::3 labels
- Cross-functional work and special projects: 25%
 - Backlog DRI: Senior Manager, PDI (based on the recommendations from the product analysts)

The distribution of work will vary from iteration to iteration, but the 50/25/25 breakdown is the target state.

Urgent issues

If an urgent matter comes up, please open an issue and tag your analyst contact (and/or @cbraza).

Please include why the issue is urgent, when it is needed by, what it will inform or how it will be used, and who is the intended audience.

If you have not heard from the tagged analyst within 1 business day (or earlier if the issue requires a faster turn-around), please send a message in data and feel free to tag @cbraza.

Please keep in mind that we work across different time zones

Additional considerations

Please keep the following in mind when working with the Product Data Insights team:

Scope creep

Scope creep is a problem everyone faces. Please keep in mind that team capacity is a zero-sum game, so scope creep in one issue can mean that we are unable to complete other work planned for that iteration.

Additional scope (change in requirements, additional follow-ups, etc) that adds a material amount of work to an issue will need to be captured as net-new work in a new issue. The new issue will